🏛️ CATÁLOGO DE LAS 7 ARQUITECTURAS (1 POR GRUPO)

📌 GRUPO 1: Arquitectura Monolítica en Capas (Traditional N-Tier / MVC)

Grupo: Maria Paz

- **Concepto:** La arquitectura clásica acoplada. Divide la aplicación en capas horizontales rígidas:

  Presentación (UI/Vista) ->Lógica de Negocio (Servicio) ->Acceso a Datos (Base de Datos).

- **Ventaja Principal:** Sencillez inicial, velocidad para prototipos y baja curva de aprendizaje.
- **El Desafío del Grupo:** Explicar por qué es ideal para iniciar un proyecto pero por qué tiende a convertirse en una "bola de barro" (*Big Ball of Mud*) cuando el sistema crece sin disciplina.

🧅 GRUPO 2: Arquitectura Cebolla (Onion Architecture)

Grupo: Gianluca

- **Concepto:** Creada por Jeffrey Palermo. Organiza el sistema en círculos concéntricos alrededor del **Modelo del Dominio**. Las capas internas no conocen nada de la infraestructura externa ni de las bases de datos.
- **Ventaja Principal:** Altamente testeable. Si cambias la base de datos o el framework, el núcleo de negocio no se ve afectado.
- **El Desafío del Grupo:** Explicar cómo la *Inversión de Dependencias (DIP)* permite que la base de datos dependa del dominio y no al revés.

🧼 GRUPO 3: Arquitectura Limpia (Clean Architecture)

Grupo: Milagros

- **Concepto:** Formulada por Robert C. Martin ("Uncle Bob"). Combina lo mejor de la Hexagonal y Onion. Se centra en separar las *Reglas de Negocio Empresariales (Entidades)* de las *Reglas de Negocio de la Aplicación (Casos de Uso / Use Cases).*
- **Ventaja Principal:** Mantenibilidad a largo plazo en grandes proyectos corporativos.

- **El Desafío del Grupo:** Mostrar la estructura completa de carpetas/capas de una aplicación y cómo fluyen los datos mediante DTOs (Data Transfer Objects).

🍕 GRUPO 4: Arquitectura de Rebanadas Verticales (Vertical Slice Architecture)

Grupo: Ricardo

- **Concepto:** Impulsada por Jimmy Bogard. Rechaza las capas horizontales globales. En su lugar, divide el código en "rebanadas verticales" independientes por cada **Caso de Uso o Funcionalidad** (ej. `RegistrarUsuario`, `PagarFactura`).
- **Ventaja Principal:** Alta cohesión y bajo acoplamiento entre funciones. Cambiar la lógica de una pantalla no afecta a ninguna otra parte del sistema.
- **El Desafío del Grupo:** Explicar cómo reemplaza los controladores y servicios genéricos por la combinación de *Commands, Queries y Handlers* (CQRS simple).

⚡ GRUPO 5: Arquitectura Orientada a Eventos (Event-Driven Architecture - EDA)

Grupo: Geovanny

- **Concepto:** El sistema reacciona a "sucesos" ocurridos en el negocio. Los componentes emiten eventos (Productores) hacia un intermediario (*Broker* como RabbitMQ, Apache Kafka o colas de Laravel) y otros servicios independientes (Consumidores) reaccionan de forma asíncrona.
- **Ventaja Principal:** Escalabilidad masiva y procesamiento asíncrono no bloqueante.
- **El Desafío del Grupo:** Explicar el flujo de un evento (ej. `PedidoRealizadoEvent`) y cómo se procesa en segundo plano sin ralentizar la respuesta al usuario.

🧩 GRUPO 6: Arquitectura de Microservicios (Microservices Architecture)

Grupo: Marco

- **Concepto:** Un sistema grande se divide en múltiples servicios pequeños, autónomos y desplegables de forma independiente. Cada microservicio es dueño de su propia base de datos aislada y se comunican por API REST, gRPC o Eventos.

- **Ventaja Principal:** Escalabilidad independiente por módulo y tolerancia a fallos.
- **El Desafío del Grupo:** Explicar los desafíos de esta arquitectura (complejidad de red, consistencia eventual de datos y monitoreo distribuido) comparados con un monolito.

⬡ GRUPO 7: Arquitectura Hexagonal (Ports & Adapters)

Grupo: Jazmin

- **Concepto:** Creada por Alistair Cockburn. Concibe la aplicación como un núcleo central (el hexágono) rodeado de **Puertos** (interfaces) y **Adaptadores** (implementaciones concretas como controladores HTTP, MySQL, APIs externas, CLI).
- **Ventaja Principal:** Desacoplamiento total de las tecnologías externas.
- **El Desafío del Grupo:** Explicar la diferencia entre un *Puerto de Entrada* (Primary/Driving) y un *Puerto de Salida* (Secondary/Driven) usando un ejemplo real en PHP/Laravel.